# YiRage (YPK) — RuntimeFusion Architecture

Engine owns schedule + KV; RF owns fusion identity; BackendRegistry searches and executes the same backend

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

**PROJECT OVERVIEW**

YiRage is a multi-backend LLM kernel compiler and runtime centered on graph superoptimization and RuntimeFusion.

It searches backend-specific fusion plans offline, then injects verified FusionCapsules into vLLM or SGLang decode steps.

## INFERENCE ENGINE · scheduling owner

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

## RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up → down
- **Native Fallback** — skip → engine native ops

## SEARCH & PLAN · offline / warm-up

- **superoptimize** — KN / TB graph search
- **FusionPlan** — μGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — optional distributed search
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

fallback

## PYTHON API & BACKEND MANAGER

- **KNGraph / TBGraph** — transpiler → kernels
- **BackendRegistry** — search + execute · same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — optional

## BACKEND IMPLEMENTATIONS

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

## HARDWARE TARGETS

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

Primary path: decode metadata → RF.step → FusionCapsule → BackendRegistry → selected backend and hardware.